

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CÔNG NGHỆ THÔNG TIN 1



NHẬT KÝ TUẦN 1
THỰC TẬP TỐT NGHIỆP

Họ và tên : Phạm Văn Kiên
Mã sinh viên : B22DCKH063
Lớp : D22CQKH01-B

Giảng viên hướng dẫn: Nguyễn Tất Thắng

Hà Nội – 2026

1. Thông tin chung

Thông tin	Nội dung
Họ và tên	Phạm Văn Kiên
Mã sinh viên	B22DCKH063
Lớp	D22CQKH01-B
Giảng viên hướng dẫn	Nguyễn Tất Thắng
Đơn vị thực tập	Học viện Công nghệ Bưu chính Viễn thông
Thời gian thực tập	Tháng 7/2026 -- nay
Thời gian báo cáo	Tuần 1
Mục tiêu tuần 1	Tìm hiểu RAG, Agentic RAG, bài toán tư vấn bán hàng và lựa chọn công nghệ

2. Mục tiêu tuần 1

Theo kế hoạch đã đề ra trong đề cương, tuần 1 tập trung vào việc xây dựng nền tảng lý thuyết và định hướng kỹ thuật cho toàn bộ đề tài. Cụ thể, tuần 1 cần hoàn thành các mục tiêu sau:

- Tìm hiểu sâu về RAG (Retrieval-Augmented Generation): nguyên lý, pipeline, thành phần và các thách thức.
- Tìm hiểu Agentic RAG: sự khác biệt so với RAG truyền thống, kiến trúc agent, cơ chế gọi công cụ (tool use) và suy luận đa bước.
- Khảo sát bài toán tư vấn bán hàng: các tình huống hội thoại thực tế, yêu cầu nghiệp vụ và ràng buộc trong thương mại điện tử.
- Lựa chọn công nghệ phù hợp cho toàn bộ pipeline: xử lý dữ liệu, truy xuất, agent, LLM, cơ sở dữ liệu và giao diện.

3. Nội dung đã thực hiện

3.1. Tìm hiểu RAG (Retrieval-Augmented Generation)

RAG là kiến trúc kết hợp giữa truy xuất thông tin (retrieval) và sinh văn bản (generation), nhằm bổ sung kiến thức riêng của tổ chức cho mô hình ngôn ngữ lớn. Thay vì dựa hoàn toàn vào kiến thức đã học trong quá trình huấn luyện, LLM trong hệ thống RAG sẽ truy xuất các tài liệu liên quan từ cơ sở dữ liệu riêng và sử dụng chúng như ngữ cảnh để sinh câu trả lời chính xác hơn.

Pipeline RAG chuẩn bao gồm các bước chính:

1. Thu thập và làm sạch dữ liệu: tài liệu sản phẩm, chính sách bán hàng, FAQ được thu thập và loại bỏ nhiễu.
2. Chunking: chia tài liệu thành các đoạn văn bản nhỏ có kích thước và ý nghĩa phù hợp.

3. Embedding: chuyển mỗi đoạn văn bản thành vector số học biểu diễn ngữ nghĩa.
4. Lưu trữ vector: các vector được lưu vào vector database (ChromaDB, FAISS, Qdrant, v.v.).
5. Retrieval: khi có truy vấn, hệ thống tìm các đoạn văn bản gần nghĩa nhất với truy vấn.
6. Augmentation: kết hợp truy vấn và các đoạn truy xuất được vào prompt.
7. Generation: LLM sinh câu trả lời dựa trên prompt đã được làm giàu.

Các thành phần then chốt trong pipeline trên:

- Embedding model: có nhiệm vụ chuyển đổi văn bản thành vector. Mô hình phổ biến là Sentence Transformers (all-MiniLM-L6-v2, all-mpnet-base-v2) hoặc các mô hình đa ngôn ngữ.
- Vector database: lưu trữ và tìm kiếm vector hiệu quả, hỗ trợ các thuật toán approximate nearest neighbor (ANN) như HNSW.
- LLM: mô hình ngôn ngữ lớn chịu trách nhiệm hiểu ngữ cảnh và sinh câu trả lời tự nhiên.
- Reranker (tùy chọn): xếp hạng lại các kết quả truy xuất để chọn đoạn văn bản thực sự liên quan nhất.

Ưu điểm của RAG:

- Giảm hallucination: câu trả lời được neo vào dữ liệu thực tế thay vì kiến thức sai lệch của LLM.
- Cập nhật nhanh: dữ liệu mới chỉ cần thêm vào vector database, không cần huấn luyện lại mô hình.
- Có thể truy xuất nguồn: người dùng có thể kiểm chứng câu trả lời qua các tài liệu tham khảo.

Hạn chế của RAG truyền thống:

- Khó xử lý các truy vấn phức tạp đòi hỏi nhiều bước, ví dụ “so sánh laptop A và B theo cấu hình và giá”.
- Khó tương tác với các công cụ bên ngoài như API tìm kiếm, lọc sản phẩm theo thuộc tính.
- Nếu truy xuất không chính xác, câu trả lời sẽ bị lệch hoặc thiếu thông tin.

3.2. Tìm hiểu Agentic RAG

Agentic RAG là mở rộng của RAG, trong đó một agent thông minh được đặt ở trung tâm để điều phối toàn bộ quy trình truy xuất và sinh câu trả lời. Agent không chỉ thực hiện một lần truy xuất mà có thể lập kế hoạch, gọi nhiều công cụ, phân tích kết quả trung gian và điều chỉnh hành động dựa trên phản hồi.

Kiến trúc điển hình của Agentic RAG bao gồm:

- Agent core: mô hình ngôn ngữ lớn đóng vai trò “bộ não”, chịu trách nhiệm lập kế hoạch và ra quyết định.
- Tool registry: tập hợp các công cụ mà agent có thể gọi, như tìm kiếm sản phẩm, lọc theo giá, so sánh sản phẩm, tra cứu chính sách.
- Memory: lưu trữ lịch sử hội thoại, ngữ cảnh hiện tại và các kết quả trung gian.
- Retriever: module truy xuất dữ liệu từ vector database hoặc cơ sở dữ liệu quan hệ.
- Planner: xác định trình tự các hành động cần thực hiện để trả lời truy vấn.

Cơ chế suy luận phổ biến là ReAct (Reasoning + Acting):

- Thought: agent phân tích truy vấn và xác định cần làm gì tiếp theo.
- Action: agent chọn và gọi công cụ phù hợp.
- Observation: agent nhận kết quả từ công cụ và đánh giá.
- Lặp lại các bước trên cho đến khi đủ thông tin để đưa ra câu trả lời cuối cùng.

So sánh RAG truyền thống và Agentic RAG:

Tiêu chí	RAG truyền thống	Agentic RAG
Luồng xử lý	Một lần truy xuất và sinh câu trả lời	Nhiều bước suy luận, có thể gọi nhiều công cụ
Tương tác công cụ	Hạn chế, chủ yếu truy xuất văn bản	Gọi đa dạng công cụ: tìm kiếm, lọc, so sánh, tra cứu
Xử lý truy vấn phức tạp	Kém hiệu quả	Hiệu quả hơn nhờ phân tích và lập kế hoạch
Tính minh bạch	Có thể hiển thị nguồn truy xuất	Có thể hiển thị cả luồng suy luận và nguồn
Độ phức tạp triển khai	Thấp	Cao hơn do cần quản lý trạng thái và tool
Chi phí/latency	Thấp	Cao hơn do nhiều lần gọi LLM/tool

Với bài toán tư vấn bán hàng, Agentic RAG phù hợp hơn vì người dùng có thể đặt các câu hỏi đa dạng: tìm kiếm, so sánh, tra cứu chính sách, kết hợp nhiều điều kiện. Agent có thể phân tích truy vấn, gọi đúng công cụ và tổng hợp câu trả lời cuối cùng.

3.3. Khảo sát bài toán tư vấn bán hàng

Trong thương mại điện tử, đặc biệt là lĩnh vực laptop và phụ kiện máy tính, người dùng thường có nhu cầu tìm hiểu kỹ trước khi mua hàng. Các chatbot truyền thống hoạt động theo kịch bản cố định, khó đáp ứng các câu hỏi tự nhiên, đa dạng. Mô hình ngôn ngữ lớn tuy linh hoạt nhưng nếu không có dữ liệu riêng sẽ dễ đưa ra thông tin thiếu căn cứ hoặc lỗi thời.

Các nhóm tình huống hội thoại phổ biến được phân loại như sau:

- Nhóm 1 -- Tìm kiếm theo nhu cầu/ngân sách: “Tôi cần laptop văn phòng dưới 15 triệu”, “Chuột không dây tầm 500k cho game thủ”.
- Nhóm 2 -- Hỏi chi tiết sản phẩm: “Laptop Dell Inspiron 15 có RAM bao nhiêu?”, “Pin sản phẩm này dùng được mấy tiếng?”.
- Nhóm 3 -- So sánh sản phẩm: “So sánh Dell XPS 13 và MacBook Air M2”, “Chuột Logitech G502 và Razer DeathAdder khác nhau thế nào?”.
- Nhóm 4 -- Tra cứu chính sách: “Chính sách bảo hành laptop là gì?”, “Đổi trả trong bao lâu?”, “Vận chuyển mất bao lâu?”.
- Nhóm 5 -- Gợi ý/giải pháp: “Tôi là sinh viên lập trình, nên chọn laptop nào?”, “Mua bộ combo bàn phím + chuột nào phù hợp văn phòng?”.

Yêu cầu chức năng của hệ thống:

- Hiểu ngôn ngữ tự nhiên tiếng Việt (hoặc tiếng Anh) và trích xuất ý định người dùng.
- Truy xuất thông tin sản phẩm, chính sách và FAQ chính xác.
- Hỗ trợ tìm kiếm, lọc, lấy chi tiết, so sánh sản phẩm.
- Sinh câu trả lời tự nhiên, có căn cứ và hiển thị nguồn tham khảo.
- Giao diện chat thân thiện, hiển thị sản phẩm gợi ý và lịch sử hội thoại.

Yêu cầu phi chức năng:

- Thời gian phản hồi nhanh, tối ưu cho trải nghiệm chat thời gian thực.
- Độ chính xác cao, giảm thiểu hallucination và câu trả lời sai lệch.
- Dễ mở rộng: dễ thêm sản phẩm mới, chính sách mới mà không cần sửa code nhiều.
- Dễ triển khai demo: kiến trúc gọn nhẹ, có thể chạy trên máy cá nhân hoặc server thử nghiệm.

3.4. Lựa chọn công nghệ

Dựa trên khảo sát lý thuyết và yêu cầu bài toán, em đã đánh giá các công nghệ ở từng thành phần và đề xuất:

Thành phần	Lựa chọn	Lý do
Ngôn ngữ	Python	Python -- thư viện AI/ML đầy đủ, cộng đồng lớn, dễ triển khai prototype.
Backend	FastAPI	FastAPI -- hiệu năng cao, async, tự động sinh API

		docs, phù hợp microservice AI.
Vector DB	ChromaDB	ChromaDB -- dễ cài đặt, hỗ trợ metadata, embedded, phù hợp cho demo và hệ thống vừa.
Embedding	Embedding model chạy local trên Ollama / API	Cân bằng giữa chất lượng, chi phí và độ trễ.
LLM	LLM API (Gemini/OpenAI)	Cân bằng giữa chất lượng, chi phí và độ trễ, dễ tích hợp.
Agent framework	LangChain / LangGraph	LangChain / LangGraph -- hỗ trợ định nghĩa tool, quản lý memory và xây dựng ReAct workflow.
Giao diện	Nodejs	Phù hợp
Product DB	SQLite	SQLite cho prototype, chuyển PostgreSQL nếu triển khai thực tế.

3.5. Thiết kế kiến trúc hệ thống dự kiến

Dựa trên các lựa chọn công nghệ trên, kiến trúc tổng quan của hệ thống được đề xuất như sau:

Người dùng → Giao diện chat → Backend API (FastAPI) → Agent điều phối (LangChain/LangGraph) → Các công cụ (tìm kiếm sản phẩm, lọc, so sánh, tra cứu chính sách) → Product DB / Vector DB → LLM → Câu trả lời tự vấn có kèm nguồn tham khảo.

Luồng xử lý khi người dùng gửi câu hỏi:

1. Giao diện gửi câu hỏi đến Backend API.
2. Agent phân tích ý định và xác định cần gọi công cụ nào (tìm kiếm, lọc, so sánh, tra cứu chính sách).
3. Agent gọi công cụ tương ứng, truy xuất dữ liệu từ Product DB hoặc Vector DB.
4. Kết quả truy xuất được đưa vào prompt cùng với lịch sử hội thoại.
5. LLM sinh câu trả lời tự nhiên, có căn cứ và kèm nguồn tham khảo.
6. Câu trả lời được trả về giao diện kèm danh sách sản phẩm gợi ý (nếu có).

Các công cụ dự kiến của agent:

- `search_products(query, category, price_range)`: tìm kiếm sản phẩm theo từ khóa, danh mục và khoảng giá.
- `get_product_details(product_id)`: lấy thông tin chi tiết của một sản phẩm.
- `compare_products(product_ids)`: so sánh nhiều sản phẩm theo các thuộc tính chính.
- `get_policy(policy_name)`: tra cứu chính sách bảo hành, đổi trả, vận chuyển, thanh toán.
- `get_faq(question)`: tra cứu câu hỏi thường gặp.

4. Kết quả đạt được

- Nắm vững nguyên lý RAG: pipeline, thành phần, ưu nhược điểm và thách thức.
- Hiểu rõ Agentic RAG, cơ chế ReAct, vai trò của agent và tool use.
- Phân loại được các nhóm tình huống tư vấn bán hàng và yêu cầu chức năng/phi chức năng.
- Đánh giá và lựa chọn được stack công nghệ chi tiết cho toàn bộ hệ thống.
- Đề xuất được kiến trúc tổng thể và danh sách công cụ của agent.

5. Khó khăn, thách thức và giải pháp

- Khó khăn: Dữ liệu sản phẩm tiếng Việt chưa có sẵn, cần tự thu thập hoặc xây dựng. Giải pháp: tìm kiếm dataset từ Kaggle, Hugging Face hoặc tự xây dựng bộ dữ liệu mẫu cho laptop và phụ kiện.
- Khó khăn: Đánh giá chất lượng embedding model trên miền sản phẩm máy tính. Giải pháp: thử nghiệm nhiều mô hình, xây dựng bộ truy vấn test và đo độ tương đồng / top-k accuracy.
- Khó khăn: Thiết kế agent đa bước dễ sai sót, loop vô hạn. Giải pháp: giới hạn số bước tối đa, thêm cơ chế dừng khi đủ thông tin, viết prompt rõ ràng.

6. Kế hoạch tuần 2

Theo đề cương và tiến độ hiện tại, tuần 2 sẽ tập trung chuẩn bị dữ liệu:

- Thu thập dữ liệu sản phẩm: laptop và phụ kiện máy tính (tên, giá, thương hiệu, cấu hình, mô tả).
- Xây dựng bộ tài liệu chính sách bán hàng: bảo hành, đổi trả, vận chuyển, thanh toán.
- Xây dựng bộ câu hỏi FAQ thường gặp và câu trả lời chuẩn.
- Làm sạch, chuẩn hóa dữ liệu và lưu trữ dưới dạng phù hợp để xử lý ở tuần 3.

7. Kết luận

Tuần 1 đã hoàn thành đầy đủ mục tiêu đề ra trong đề cương. Em đã tìm hiểu sâu về RAG và Agentic RAG, khảo sát chi tiết bài toán tư vấn bán hàng, phân tích các tình huống hội thoại thực tế, đánh giá và lựa chọn công nghệ phù hợp, đồng thời đề xuất kiến trúc tổng thể cho hệ thống. Những kết quả này là cơ sở vững chắc để triển khai các công việc tiếp theo: chuẩn bị dữ liệu, xử lý dữ liệu, xây dựng API, agent và giao diện.